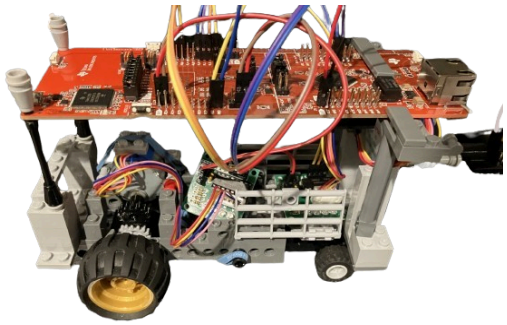
Autonomous SPATIAL MAPPING SYSTEM using Time-of-Flight

1. Device Overview



1.1 Features

- Bus Speed
  - 28 MHz PLL configured clock speed
- Operating Voltage
  - Microcontroller (MSP432E401Y): 4.75 - 5.25 V
  - ToF Sensor (VL53L1X): 2.6 - 3.5 V
  - Stepper Motor with Driver (28BYJ-48 + ULN2003): 5 V
- Cost
  - TI MSP432E401Y Microcontroller: $250 CAD
  - VL53L1X ToF Sensor Module: $20 CAD
  - Stepper Motor + Driver (28BYJ-48 + ULN2003) x2: $10 CAD
- Serial Communication
  - I2C (Sensor Communication)
    - Speed: Standard 100 kHz
  - UART (PC Communication)
    - Baud Rate: 115200 bps
    - Data Format: 8 data bits, no parity, 1 stop bit (8N1)
- Programming Language
  - Python (Libraries: PySerial, NumPy, Open3D)
- Memory
  - MSP432E401Y (Flash: 1 MB | SRAM: 256 KB)

1.2 General Description

This embedded spatial mapping system uses a time-of-flight (ToF) sensor to acquire information about the surrounding environment. Pressing the onboard button PJ0 initiates the mapping process.

Distance measurements are obtained using the VL53L1X ToF sensor, which is mounted on a stepper motor to scan a 360° vertical (y-z) plane. A second stepper motor moves the vehicle forward along the x-axis by a fixed distance between scans, allowing the system to build a full 3D representation of the environment.

The VL53L1X sensor integrates all aspects of distance acquisition and signal processing in a single package: it detects the target, converts the non-electrical distance signal to an electrical signal, conditions it for analog to digital conversion, and then outputs a discrete digital distance measurement via I2C to the microcontroller.

The microcontroller then transmits the distance data to a PC using UART serial communication at 115200 bps. Start and end flags are used to ensure proper synchronization of the data transmission so that measurements are not lost.

Finally, on the PC, a Python script captures and parses the incoming UART data. Each distance measurement is converted to x, y, z coordinates using trigonometry to generate a point cloud, which is then graphically visualized to create a 3D map of the scanned environment.

1.3 Block Diagram

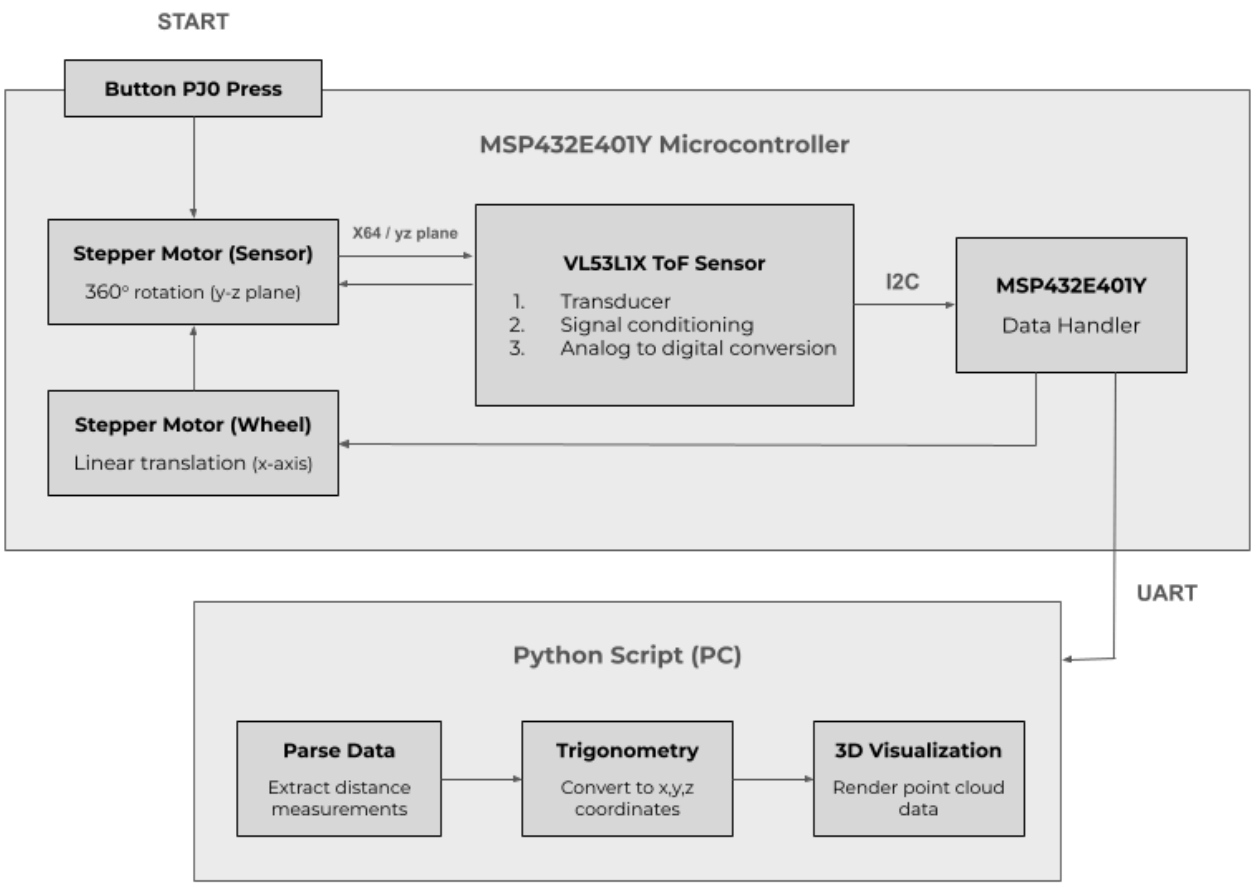


Figure 1: Block Diagram of Spatial Mapping System

2. Device Characteristics

Table 1: Device Characteristics

MSP432E401Y Microcontroller	
Bus Speed	28 MHz
Onboard Button	PJ0
Measurement Status LED	PF4
UART Tx Status LED	PN1
Vehicle Driving Status LED	PN0
ULN2003 Motor Driver (Sensor Location)	
V+	5V
V-	GND
IN0 : IN3	PL0 : PL3
ULN2003 Motor Driver (Wheel Location)	
V+	5V
V-	GND
IN0 : IN3	PH0 : PH3
VL53L1X ToF Sensor	
Vin	3.3 V
GND	GND
SCL	PB2
SDA	PB3
I2C Communication Speed	100 kHz (standard)
Sensor Device Address	0x29
Distance Mode	4 m (long)
Timing Budget	100 ms
UART Communication	
Serial Port	UART0 (PJ4 and PJ5 jumpered in vertical orientation)
Baud Rate	115200 bps

3. Detailed Description

3.1 Distance Measurement: Acquisition

Using interrupt-based programming, the system waits for the onboard button (PJ0) to be pressed before initiating the scanning process. Once triggered, the stepper motor performs a full 360° rotation, pausing every 5.625° to capture a distance measurement, resulting in a total of 64 measurements per y-z slice. The status LED (PF4) flashes each time a measurement is successfully acquired.

The VL53L1X ToF sensor operates by emitting a 940 nm infrared laser pulse and detecting the reflected signal using a SPAD array. By measuring the time delay between emission and detection and using the speed of light, the distance can be determined using the formula: $d = vt / 2$. This process is handled internally by the sensor, including transduction, signal conditioning, and analog to digital conversion, with the final digital distance output provided in millimeters. Each measurement is represented as a 16-bit value and transmitted to the microcontroller via I2C.

After completing a full 360° scan of a y-z slice, the 64 distance measurements are transmitted to the PC over UART, indicated by a flash of the status LED (PN1). The second stepper motor is then activated to drive the rear wheels of the vehicle, with LED (PN0) turning on to indicate motion. With a 1:1 gear ratio and a tire circumference of 14 cm, one full motor revolution produces a 14 cm displacement along the *x*-axis. The motor is programmed to complete two full revolutions, resulting in a total displacement of 28 cm (approximately 1 ft).

Following this movement, the first stepper motor is reactivated and the scanning process repeats for the next y-z slice. This cycle continues for a user-defined number of iterations, specified through a configurable software parameter to capture the entire environment as multiple snapshots along the x-axis.

3.2 Distance Measurement: Data Processing

On the PC, a Python script captures and parses the incoming UART data using the PySerial library. After a “START” flag is received from the microcontroller, the script reads a full set of distance measurements for one 360° scan using the *collect_scan()* helper function, where the measurements are stored in a global list. Once all the scans are collected, indicated by a terminating “END” flag over UART, the data is processed in the main program. For each scan, the *convert_xyz()* helper function is called to convert the list of raw distance data into a x,y,z Cartesian coordinates.

Within the function *convert_xyz()*, each distance measurement is first converted from millimeters to meters by dividing the value by 1000 so that everything is in S.I. units. The corresponding angle for each measurement is then generated using evenly spaced values over the range of 2π (360°), creating a list of values with an equivalent step size of 5.625°. The Cartesian coordinates are then computed using the NumPy library and the trigonometric relationships:

$$x = x_{coord}$$

$$y = d * sin(\theta)$$

$$z = d * cos(\theta) + h$$

where d is the measured distance, θ is the scan angle in radians, and h is the sensor height above the ground (0.068 m). The *x*-coordinate remains constant for each scan and is incremented by 0.28 m between scans to account for the forward motion of the vehicle.

For example, if a measured distance is 500 mm (0.5 m) at an angle of 0°in the first y-z slice, the coordinates are calculated as:

$$x = 0$$

$$y = 0.5 * sin(0) = 0$$

$$z = 0.5 * cos(0) + 0.068 = 0.568\ m$$

Final converted x,y,z coordinate = [0, 0, 0.568]

All computed coordinates are stored as a point cloud and written to a .xyz file, where each row represents a single point in 3D space. Writing the coordinates to a file ensures the data is preserved and can be revisualized by Open3D at a later time.

3.3 Visualization

Within the Python script, the function *visualize_data()* uses the library Open3D to create and render the 3D map from the previously stored scan data. It loads the saved .xyz file using *o3d.io.read_point_cloud(),* extracts the point coordinates into a NumPy array, and then computes the line connectivity.

For each y-z slice, the function connects each point to the next point in the scan, using wraparound modulo logic so that the slice is closed into a full 360° loop. It also connects each point in one scan to the corresponding point in the next scan to illustrate how the environment changes along the x-axis as the vehicle moves forward.

These line connections are stored as index pairs and passed into the function *o3d.geometry.LineSet(points, lines),* which associates the line segments with the 3D coordinates. Finally, the function *o3d.visualization.draw_geometries()* renders the result as a wireframe-style visualization to show each scan ring and the overall spatial structure formed by multiple slices, thus recreating the scanned environment.

4. Application

4.1 Application Note

A primary application for this system is indoor exploration and navigation for small-scale autonomous robots. While commercial Light Detection and Ranging (LIDAR) equipment is often the industry standard for spatial mapping, their high cost and physical bulk make them impractical for small, budget-sensitive applications. By utilizing a Time-of-Flight (ToF) sensor paired with a precision stepper motor, this system provides a small, lightweight, and cost-effective alternative for generating 3D representations of enclosed environments.

4.2 Instructions

The following instructions assume that the user has already installed the Keil uVision5 software and the necessary Python libraries (PySerial, NumPy, Open3D).

1. Physically assemble the system by following the pin assignment in Table 1 or the circuit schematic in Figure 4.
2. Plug the micro-USB cord into the XDS110 debug port on the microcontroller and connect the other end to the PC.
3. Open the Device Manager on the PC, look under Ports (COM & LPT), find the XDS110 Class Application/ User UART, and note the port number.
4. Open the Python script and under the configuration heading, change the serial port to the specific COM port identified in Step 2.
5. Open the Keil uVision5 software and open the project named “Final_Project_2DX3”.
6. Set the macro *NUM_SCANS* to the desired number of scans for mapping the environment.
7. Click the “Options for Target” button.
    a. Under C/C++, ensure optimization is set to “-O0”.
    b. Under Debug, ensure use is set to “CMSIS-DAP Debugger” and the SW device is present in the settings tab.
8. Press File -> Save.
9. Translate, build, and flash load the code onto the microcontroller. Press the onboard reset button once all 3 processes are complete.
10. Wait for all 4 onboard LEDs to finish flashing twice. The system is now initialized and ready for operation.
11. Run the Python script. A verification message will be printed to the terminal if the serial port is successfully opened.
12. Start the scanning process by pressing the onboard button PJ0.
13. After the specified number of scans have been completed, a pop-up window from Open3D will appear with the visualization of the mapped environment.

4.3 Expected Output

The expected output of the system can be seen through the side-by-side comparison of a campus hallway and the 3D model generated based on the scanned data of the location.

Figure 2: Engineering Technology Building (ETB) - Second Floor - Hallway G

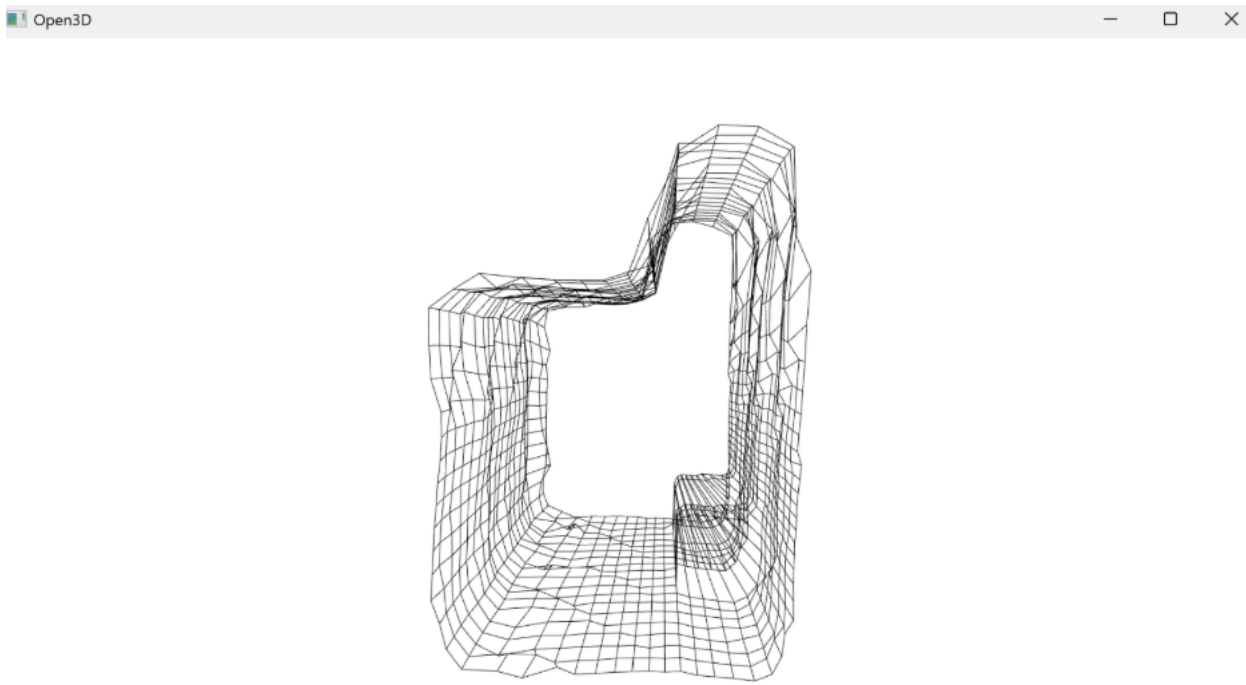


Figure 3: Visualization of Hallway G

5. Limitations

5.1 Microcontroller

While coordinate conversion is handled on the PC during data processing, if the calculations were instead performed on the MSP432E401Y microcontroller, they would be subject to several limitations.

The MSP432E401Y contains an ARM Cortex-M4F core equipped with a single-precision Floating Point Unit (FPU), meaning it can only perform floating-point operations up to 7 significant digits of precision. Any result requiring more decimal places than this gets rounded off, introducing numerical error.

Additionally the trigonometric functions such as sin() and cos(), used to convert the distance and angle data into x, y, z coordinates, often produce irrational values with infinitely many decimal places. The <math.h> library would need to be used to perform these calculations, and it must truncate the trigonometric values at some point to return a usable result, meaning every sin() and cos() output already carries a small rounding error before any further calculation is done.

When combined, these two limitations of FPU capabilities and trigonometric functions worsen the overall coordinate accuracy. A trigonometric result that is already slightly rounded gets multiplied by a distance value, causing the error to carry through and grow. At longer distances, this translates to significant positional errors in the final 3D point cloud.

5.2 Communication Speed with ToF Modules

As required per the project specifications, a bus frequency of 28MHz was needed for the microcontroller. To achieve this, the PLL was configured using the following parameters and formula:

$VCO = MOSC\ (f_{SRC} = 25MHz)$

$Q = 0$

$N = 4$

$MINT = 28$

$PSYSDIV = 4$

$SYSCLK = [\ f_{SRC} / (Q+1)(N+1)\ ] * MINT / (PSYSDIV + 1) = 28\ MHz$

Since the microcontroller communicates with the VL53L1X ToF sensor via I2C, a synchronous protocol whose clock frequency is derived from the system bus clock, the I2C configuration needed to be updated to account for the reduced bus speed of 28 MHz. To maintain the standard communication rate of 100 kHz, the Timer Period Register (MTPR) was recalculated and set to a value of 13 (0x0D) using the following parameters and formula:

$CLK_PRD = 1 / SYSCLK = 35.7\ ns$

$TIMER_PRD = 1$

$SCL_LP = 6$

$SCL_HP = 4$

$SCL\ period = 1 / 100kHz = 10\ us$

$SCL\ period = 2 * (1 + TPR) * (SCL_LP + SCL_HP) * CLK_PRD$

$10\ us = 2 * (1 + TPR) * (10) * 35.7\ ns$

$TPR = 13$

5.3 System Bottlenecks

The primary bottleneck of the system's speed is the 28BYJ-48 stepper motor. Each step requires the motor's internal teeth to physically snap into place before the next coil is energized. If the delay between steps is too short, the rotor cannot keep up and the motor stalls or skips steps. To determine the fastest permissible speed between steps, the delay was progressively reduced until the motor stalled, confirming the mechanical limit was reached. The delay ceiling for this system was found to be approximately 2 ms between steps.

A secondary bottleneck is the VL53L1X ToF sensor. It was configured to use the long distance mode (4 m) because some indoor ceilings exceed 3 m, which is beyond the range of the next closest mode. The returning laser signal becomes weaker as distance increases, so the timing budget was set to 100 ms to collect enough pulses for a reliable measurement. This 100 ms per reading directly limits how fast data can be collected, regardless of I2C clock speed. This was verified by timestamping calls before and after *VL53L1X_GetDistance()*, confirming the interval was bounded by the 100 ms timing budget rather than the 10 us I2C clock period.

6. Circuit Schematic

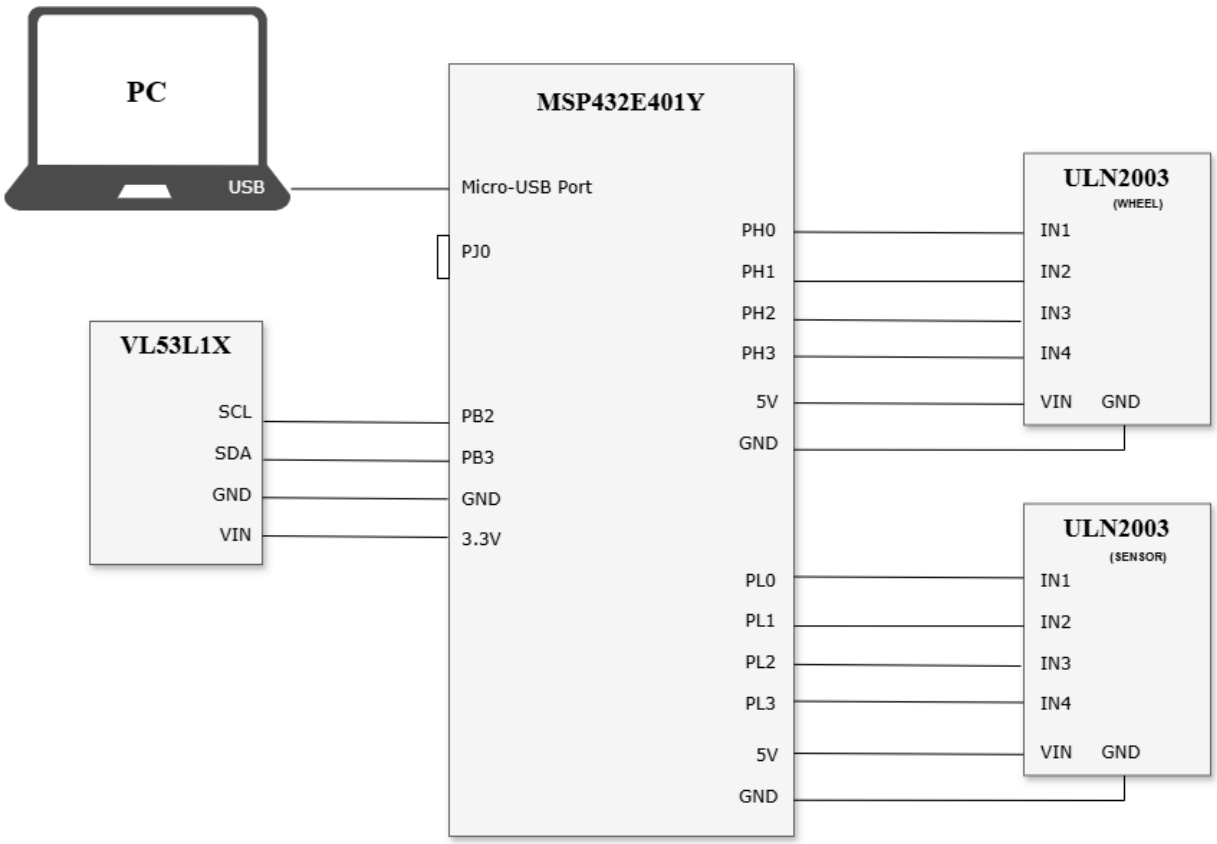


Figure 4: Spatial Mapping System Circuit Schematic

7. Programming Logic

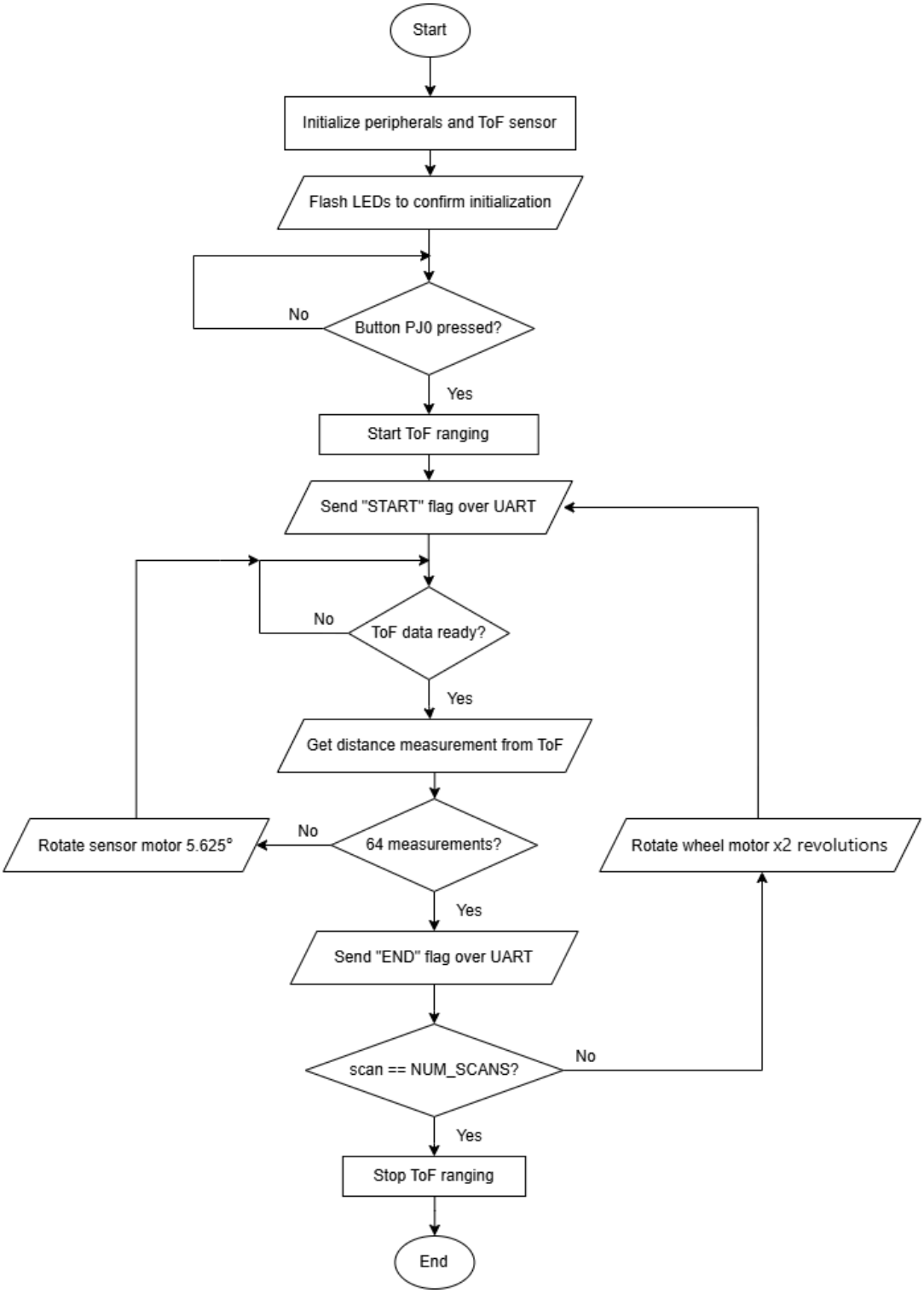


Figure 5: Flow Chart for Microcontroller Code

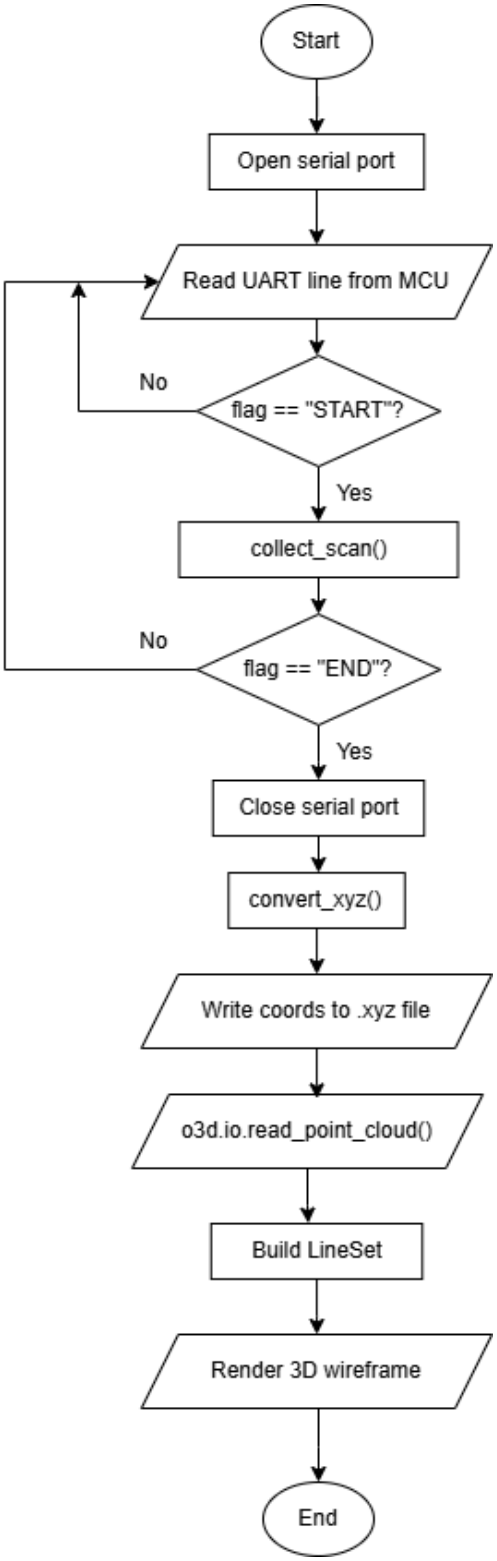


Figure 6: Flow Chart for Python (PC) Code